Multi-Agent Debate AI System

for Intelligent Decision Making

1. Project Overview

The Multi-Agent Debate AI System is a production-grade application that simulates expert panel discussions using multiple Large Language Model (LLM) powered agents. Each agent carries a unique role, reasoning style, and personality. They argue, counter each other, verify facts, and collectively produce well-reasoned decisions on complex queries.

The system is designed to replicate real-world expert decision panels — such as financial analysts, risk managers, and domain strategists — in a fully automated, explainable AI pipeline.

1.1 Core Objectives

- Deploy multiple specialised LLM agents with distinct roles and personalities
- Orchestrate structured multi-round debates (3 rounds per query)
- Incorporate Retrieval-Augmented Generation (RAG) for real-world data grounding
- Produce scored, explainable, and reliable final decisions
- Persist debate history with semantic memory for recall across sessions
- Provide a production-ready Streamlit dashboard for interaction and analysis

1.2 Supported Domains

Domain	Focus Areas	Example Query
Finance	ROI, risk, market trends, regulatory compliance	Should we invest in renewable energy stocks?
Business Strategy	SWOT, competitive advantage, market positioning	Should our startup pivot from B2C to B2B?
Policy & Ethics	Societal impact, fairness, legal frameworks	Should AI content require mandatory disclosure?
Technology	Scalability, feasibility, architecture trade-offs	Should we migrate to microservices?

2. System Architecture

The system follows a layered, modular architecture with five core layers communicating through well-defined interfaces.

2.1 Architecture Diagram

```
┌──────────────────────────────────────────────────────────────┐
│                    STREAMLIT FRONTEND                        │
│  [Query Input] → [Run Debate] → [Tabs: Debate|Analytics]     │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                ORCHESTRATION LAYER                           │
│            debate_engine.py  (core)                          │
│  Planner → Round Manager → Scoring → Judge → Output          │
└──────────────────────────────────────────────────────────────┘
     │                │                  │
     ▼                ▼                  ▼
┌─────────┐    ┌──────────┐    ┌──────────────────┐
│  AGENT  │    │   RAG    │    │   MEMORY STORE   │
│  LAYER  │    │  LAYER   │    │   (FAISS/JSON)   │
│Planner  │    │Embeddings│    │Short: session    │
│Optimist │    │FAISS Index│    │Long:  history    │
│Skeptic  │    │Retriever │    │Semantic search   │
│Analyst  │    └──────────┘    └──────────────────┘
│Expert   │         │                  │
│Critic   │         └──────────────────┘
│FactChk  │                  │
│Judge    │                  ▼
└────┬────┘    ┌──────────────────────┐
     └────────►│     LLM API LAYER    │
               │  Groq (llama3-70b/8b)│
               └──────────────────────┘
```

2.2 Layer Descriptions

Frontend Layer

Built with Streamlit, the frontend provides a query input area, domain selector, RAG file upload, and three result tabs: Debate Transcript, Analytics, and History. A sidebar handles configuration and memory browsing.

Orchestration Layer

debate_engine.py is the central coordinator. It initialises all agents, drives the 3-round debate loop, triggers the Critic and Fact-Checker after each round, aggregates scores via ScoringEngine, and invokes the Judge for a final verdict.

Agent Layer

Eight specialised agents, each inheriting from BaseAgent, communicate exclusively through the LLM API layer. They are stateless between debates but receive debate_history as context for multi-round awareness.

RAG Layer

Retrieval-Augmented Generation pipeline using SentenceTransformers (all-MiniLM-L6-v2) for free local embeddings and FAISS for similarity search. Chunks from PDFs or text are indexed before the debate and retrieved per-query.

Memory Layer

Two-tier memory system: ShortTermMemory (in-RAM dict) tracks live debate state; LongTermMemory persists completed debates to JSON files and a separate FAISS index for semantic recall of past decisions.

3. Agent Design

All agents inherit from BaseAgent and follow a strict interface: build_prompt() → call_llm() → return AgentResult. The debate engine treats all agents uniformly.

Agent	Role	LLM Mode	Temperature	Output
Planner	Structures debate agenda	_call()	0.2 (low)	Numbered agenda
Optimist	Best-case advocate	_call()	0.8 (creative)	Argument text
Skeptic	Risk & challenge finder	_call()	0.6 (precise)	Counter-argument
Analyst	Data-driven evidence	_call()	0.4 (accurate)	Evidence argument
Domain Expert	Technical depth	_call()	0.7 (balanced)	Expert opinion
Critic	Logic & quality scorer	_call_json()	0.3 (consistent)	JSON scores
Fact-Checker	Claim verification	_call_json()	0.3 (consistent)	JSON flags
Judge	Final decision maker	_call_json()	0.2 (deterministic)	JSON verdict

3.1 Debate Round Progression

- Round 1 — Opening Arguments: Each agent states their initial position using RAG context
- Round 2 — Counterarguments: Each agent reads Round 1 outputs and challenges opponents
- Round 3 — Refinement & Closing: Agents refine positions and deliver closing statements
- Post-Round (each round): Critic scores all arguments; Fact-Checker verifies claims

3.2 Scoring Metrics

The Critic scores each agent on four metrics (0–25 points each) per round:

- Logical Consistency — Is the argument free of contradictions?
- Evidence Usage — Are data, statistics, or retrieved context cited?
- Relevance — Does the argument address the debate topic directly?
- Persuasiveness — Is the argument compelling and well-structured?

Scores are weighted across rounds: Round 1 = 25%, Round 2 = 35%, Round 3 = 40%. Maximum weighted score per agent = 100 points.

4. Project File Structure

```
multi-agent-debate/
│
├── app.py                          # Streamlit entry point
├── requirements.txt                # Python dependencies
├── .env.example                    # API key template
├── .env                            # Your actual keys (not committed)
│
├── backend/
│   ├── __init__.py
│   ├── config.py                   # All constants, domain configs, thresholds
│   ├── debate_engine.py            # Core orchestrator — runs full pipeline
│   └── scoring.py                  # Aggregates Critic scores → leaderboard
│
├── agents/
│   ├── __init__.py
│   ├── base_agent.py               # Abstract base class (AgentResult, _call*)
│   ├── planner.py                  # Structures debate agenda
│   ├── optimist.py                 # Best-case advocate
│   ├── skeptic.py                  # Risk challenger
│   ├── analyst.py                  # Data-driven evidence
│   ├── domain_expert.py            # Domain-specific technical depth
│   ├── critic.py                   # Scores arguments (JSON output)
│   ├── fact_checker.py             # Verifies claims (JSON output)
│   └── judge.py                    # Final verdict (JSON output)
│
├── rag/
│   ├── __init__.py
│   ├── loader.py                   # PDF + TXT chunker
│   ├── embedder.py                 # SentenceTransformers embeddings
│   ├── vector_store.py             # FAISS index (add, search, save, load)
│   └── retriever.py                # Full RAG pipeline coordinator
│
├── memory/
│   ├── __init__.py
│   ├── memory_schema.py            # Data models for records and sessions
│   ├── short_term.py               # In-RAM session state
│   └── long_term.py                # JSON + FAISS persistent history
│
├── frontend/
│   ├── __init__.py
│   ├── ui_components.py            # Header, sidebar, decision card, history
│   ├── debate_view.py              # Debate transcript tab
│   └── analytics_view.py           # Charts, scores, token usage tab
│
├── utils/
│   ├── __init__.py
│   ├── llm_client.py               # Groq API wrapper (retry, JSON mode)
│   ├── prompt_builder.py           # Dynamic prompt construction per agent
│   └── logger.py                   # Centralised logging (console + file)
│
└── data/
    ├── vector_index/               # FAISS index files (auto-created)
    ├── debate_history/             # JSON debate records (auto-created)
    └── uploads/                    # Temp PDF uploads (auto-created)
```

5. End-to-End Workflow

Step 1 — User Input

- User enters a decision query in the Streamlit text area
- Selects domain: Finance / Business Strategy / Policy & Ethics / Technology
- Optionally uploads PDF or TXT documents for RAG context
- Clicks 'Run Debate' button

Step 2 — RAG Retrieval (if enabled)

- Uploaded files are chunked (500 chars, 50-char overlap)
- Each chunk is embedded using all-MiniLM-L6-v2 (local, free)
- Embeddings are indexed in FAISS IndexFlatIP (cosine similarity)
- Query is embedded and top-5 most relevant chunks are retrieved
- Retrieved chunks are formatted into a context string for agents

Step 3 — Planning

- Planner Agent receives query + RAG context
- Outputs a numbered agenda of 3–5 debate propositions
- Agenda is injected into all subsequent agent prompts

Step 4 — Debate Rounds (×3)

- Each round follows: Optimist → Skeptic → Analyst → Domain Expert
- Round 1: Opening arguments using RAG context
- Round 2: Counterarguments — each agent reads Round 1 outputs
- Round 3: Closing statements — refined positions and final case
- After each round: Critic scores all four agents; Fact-Checker verifies claims

Step 5 — Scoring Aggregation

- ScoringEngine applies round weights (25% / 35% / 40%)
- Computes weighted totals, per-metric averages, and leaderboard
- Identifies debate winner (highest scoring agent)

Step 6 — Final Judgment

- Judge Agent receives full transcript + scores + fact-check flags
- Outputs: final decision, confidence score (0–100%), risk level, key reasons
- Also acknowledges strongest dissenting view for balance

Step 7 — Storage & Display

- Debate saved to LongTermMemory as JSON + FAISS summary embedding
- UI renders: Decision Card, Debate Transcript tab, Analytics tab, History tab

6. Technology Stack

Category	Technology	Version	Purpose
LLM API	Groq Cloud	Latest	llama3-70b / 8b inference
LLM Models	Llama3-70B-8192	Meta	Primary debate model
LLM Models	Llama3-8B-8192	Meta	Fast scoring model
Embeddings	SentenceTransformers	2.6+	all-MiniLM-L6-v2 (local)
Vector DB	FAISS-CPU	1.7.4+	Similarity search
Frontend	Streamlit	1.32+	Web UI + charts
PDF Parsing	PyPDF	4.0+	Document loading
Data Handling	Pandas + NumPy	2.x / 1.26+	Analytics charts
HTTP Client	Requests	2.31+	Groq API calls
Config	python-dotenv	1.0+	.env key management
Persistence	JSON Files	stdlib	Debate history storage
Logging	Python logging	stdlib	Console + file logs
Deployment	Streamlit Cloud	Free tier	Hosting

6.1 Why These Choices

- Groq — Fastest free LLM API available; llama3-70b gives GPT-4 level reasoning at zero cost
- FAISS — Handles 10k+ chunks with sub-millisecond search; no server needed; pure local
- SentenceTransformers — Free local embedding model; downloads once, runs offline after
- Streamlit — Zero-configuration frontend; perfect for ML demos; free cloud hosting
- JSON for persistence — No database setup; portable; human-readable; sufficient for 100 debates

7. Setup & Run Commands

7.1 Prerequisites

- Python 3.10 or higher
- Node.js 16+ (for docx generation — not needed for the main app)
- Free Groq API key from console.groq.com
- Git

7.2 Local Setup

Clone the Repository

```
git clone https://github.com/yourusername/multi-agent-debate.git
cd multi-agent-debate
```

Create Virtual Environment

```
# Windows
python -m venv venv
venv\Scripts\activate

# macOS / Linux
python -m venv venv
source venv/bin/activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Configure API Key

```
# Copy the template
cp .env.example .env

# Edit .env and add your key
GROQ_API_KEY=your_groq_api_key_here
```

Create Required Directories

```
mkdir -p data/vector_index data/debate_history data/uploads
```

Run the Application

```
streamlit run app.py
#to remove warnings
streamlit run app.py --server.fileWatcherType none

# App opens at: http://localhost:8501
```

7.3 requirements.txt

```
streamlit>=1.32.0
requests>=2.31.0
python-dotenv>=1.0.0
faiss-cpu>=1.7.4
sentence-transformers>=2.6.0
pypdf>=4.0.0
numpy>=1.26.0
pandas>=2.2.0
```

7.4 Streamlit Cloud Deployment

- Push your repository to GitHub (ensure .env is in .gitignore)
- Go to share.streamlit.io and sign in with GitHub
- Click 'New app' → select your repository → set main file to app.py
- Open 'Advanced settings' → 'Secrets' and add:

```
GROQ_API_KEY = "your_groq_api_key_here"
```

- Click 'Deploy' — app is live in ~2 minutes on a free URL

7.5 Docker Deployment (Optional)

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.port=8501"]

# Build and run
docker build -t debate-ai .
docker run -p 8501:8501 -e GROQ_API_KEY=your_key debate-ai
```

8. Key Design Patterns

8.1 Singleton LLM Client

get_llm_client() returns a shared LLMClient instance across all 8 agents. This avoids re-initialising API connections on every agent call and centralises retry logic, error handling, and token tracking.

8.2 Abstract Base Agent

BaseAgent enforces a strict contract: every agent must implement build_prompt() and run(). This guarantees the debate engine can call any agent uniformly without agent-specific logic at the orchestration level.

8.3 Progress Callbacks

DebateEngine.run() accepts an on_progress(stage, message) callback. Streamlit passes a function that updates the UI status widget in real time. This decouples the engine from the UI framework — the engine can run headlessly in tests or scripts.

8.4 JSON-Mode Agents

The Critic, Fact-Checker, and Judge use _call_json() which forces structured JSON output and parses it safely. The parsed dict is stored in AgentResult.metadata while a human-readable string goes into AgentResult.content — both formats are available for the UI.

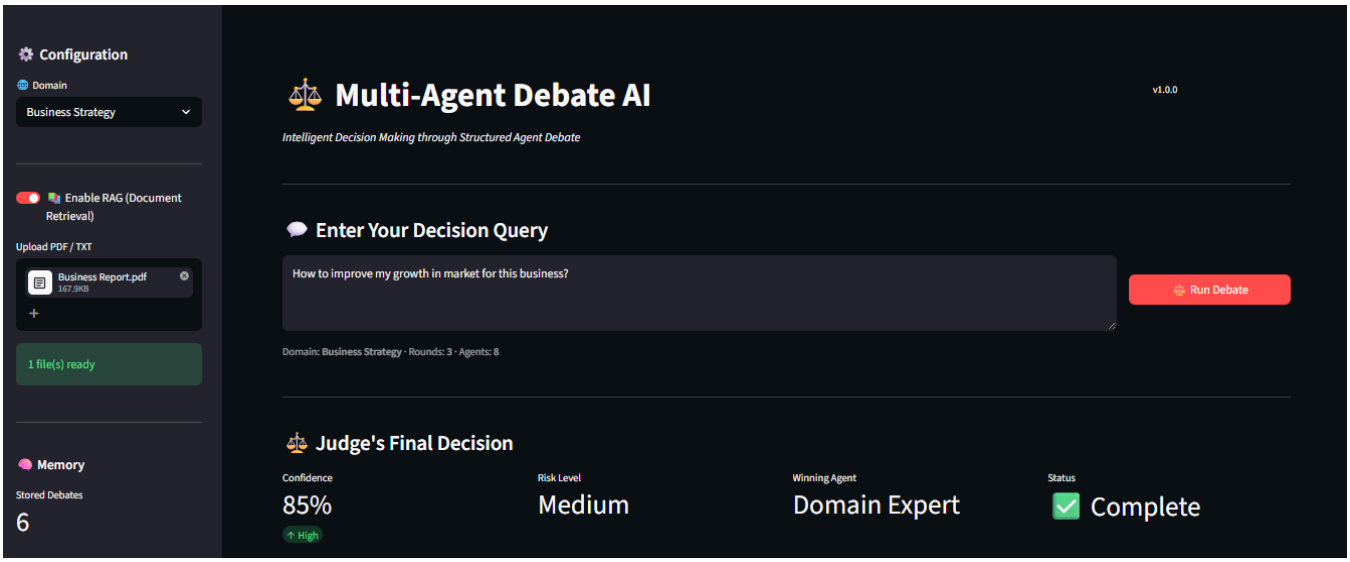
8.5 Domain Injection

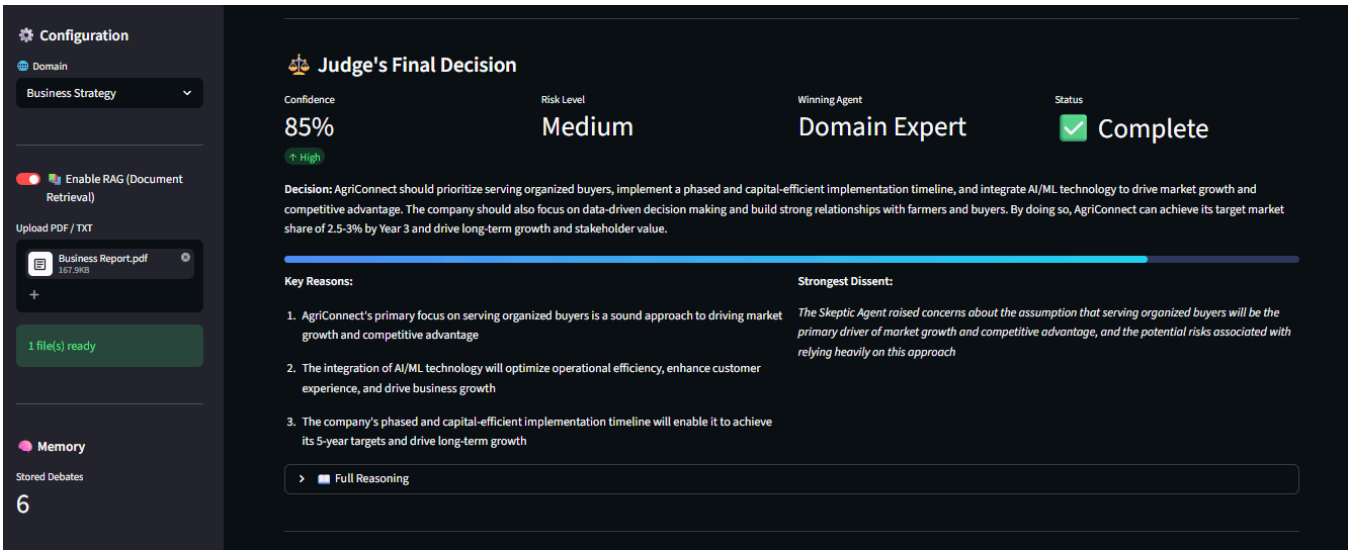
PromptBuilder takes a domain at initialisation and injects the domain context string into every agent's system prompt. The same Skeptic agent automatically behaves like a financial risk manager in Finance mode and a policy analyst in Ethics mode — no code changes needed.

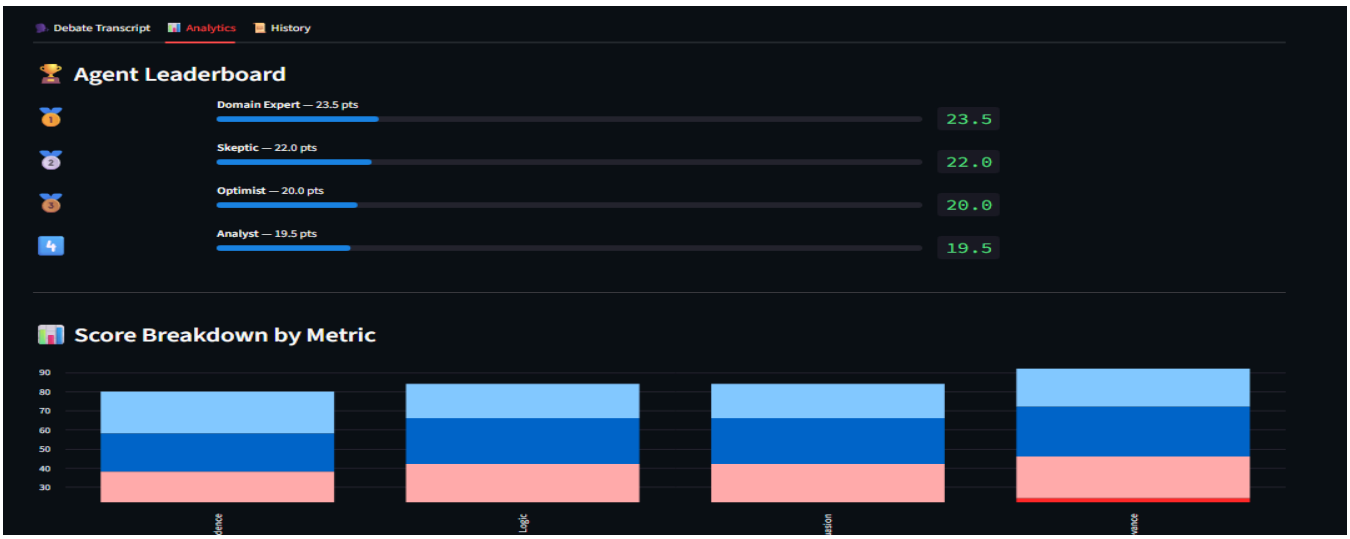
8.6 Two-Store FAISS

The system maintains two completely separate FAISS indexes: one for RAG document chunks (data/vector_index/debate.index) and one for long-term memory summaries (data/vector_index/memory.index). They never mix, preventing retrieval cross-contamination.

9. Expected System Outputs



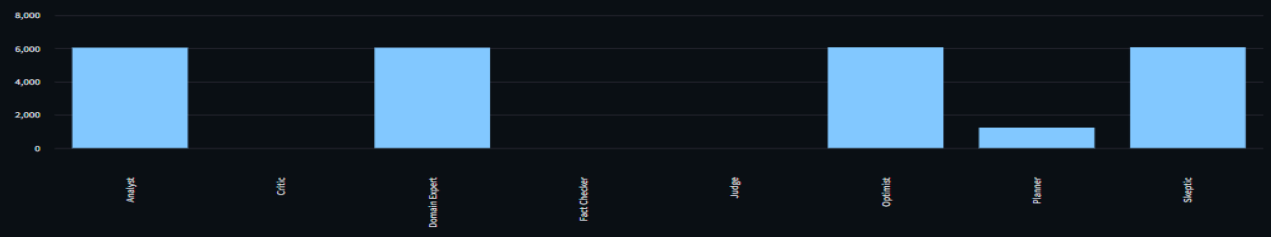




Multi-Agent Debate AI System

Deploy

25,489 | 136.81s | 3,186



Fact-Check Summary

Total Claims	Verified	Uncertain	Unverified
36	26	9	1

Factual accuracy: 72%

Raw Debate Data (JSON)

Debate Agenda

1. AgriConnect's primary focus on serving organized buyers will be the key driver of its market growth and competitive advantage in the Indian agricultural commerce sector.
2. The implementation of a phased, capital-efficient implementation timeline, leveraging proven GTM strategies, is essential for AgriConnect to achieve its 5-year target of 500,000 farmer suppliers and ₹500 crores in GMV.
3. AgriConnect's organizational structure, with a strong emphasis on data-driven decision making and a dedicated team for farmer and buyer relations, is sufficient to support its long-term growth and market share goals of 2.5-3% of the Serviceable Obtainable Market (SOM).
4. The integration of AI/ML technology into the AgriConnect platform will be a critical factor in optimizing operational efficiency, enhancing customer experience, and ultimately driving business growth and stakeholder value.
5. AgriConnect's target market share of 2.5-3% of the Serviceable Obtainable Market (SOM) by Year 3 is ambitious and achievable, given the current market trends and the company's strategic plans for market development and stakeholder relations.

Round 1 | Round 2 | Round 3

Round 1 — Arguments

Optimist

I'm thrilled to present my opening argument for why AgriConnect's market growth prospects are exceedingly bright. With a well-structured business plan, a strong organizational foundation, and a clear focus on serving organized buyers, AgriConnect is poised to capitalize on the vast opportunities in the Indian agricultural commerce sector. As outlined in the context, the company's primary focus on serving organized buyers will be the key driver of its market growth and competitive advantage. This targeted approach, combined with a phased, capital-efficient implementation timeline, will enable AgriConnect to achieve its ambitious 5-year targets of 500,000 farmer suppliers and ₹500 crores in GMV.

The evidence from the context suggests that AgriConnect has a deep understanding of the market and its needs. The company's data-driven decision-making approach, which relies on empirical insights from agricultural market data, farmer income data, and food business sourcing patterns, will enable it to navigate the market effectively. For instance, the fact that the agricultural market

Critic Scores

```
ridence: 18 | Relevance: 22 | Persuasion: 20 | Total:  80/100
ir and focused vision for AgriConnect's growth, but lacks a comprehensive risk a
ridence: 20 | Relevance: 24 | Persuasion: 22 | Total:  88/100
ntifies key risks and weaknesses in the optimistic view, but could be more cons
ridence: 22 | Relevance: 20 | Persuasion: 18 | Total:  78/100
on data-driven insights, but could benefit from more critical analysis and cons
ridence: 20 | Relevance: 26 | Persuasion: 24 | Total:  94/100
-structured and informed perspective on AgriConnect's growth prospects, drawing
```

Fact-Check Report

Optimist

VERIFIED — AgriConnect's primary focus on serving organized buyers will be the key driver of its market growth

└ Context mentions AgriConnect's focus on serving organized buyers as a key strategy.

VERIFIED — AgriConnect aims to onboard 500,000 farmer suppliers and ₹500 crores in GMV by the end of Year 3.

└ Context mentions 5-Year Target: 500,000 farmer suppliers, 15,000 buyer organizations and GMV Target: ₹500 crores by Year 3.

VERIFIED — The integration of AI/ML technology into the AgriConnect platform will optimize operational efficien

└ Context mentions AI/ML technology as a critical factor in optimizing operational efficiency and driving business growth.

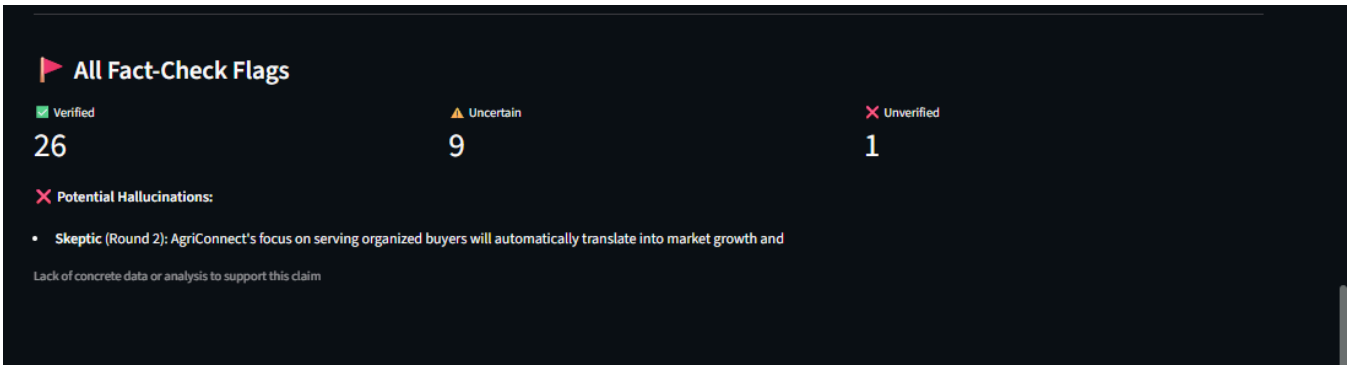
Skeptic

VERIFIED — The Serviceable Obtainable Market (SOM) is estimated to be ₹500-750 crores.

└ Context mentions SOM: ₹500-750 crores

UNCERTAIN — AgriConnect's organizational structure may not be agile enough to respond to changing market conditi

└ Context mentions organizational structure, but does not provide enough information to determine its agility.



9.1 Debate Transcript

For each of the 3 rounds, the system produces:

- Optimist argument (3–4 paragraphs, evidence-based)
- Skeptic counter-argument (challenges assumptions and risks)
- Analyst data-driven position (quantitative evidence)
- Domain Expert technical opinion (domain-specific depth)
- Critic score table (4 metrics × 4 agents = 16 scores per round)
- Fact-check report (verified ✅ / uncertain ⚠️ / unverified ❌ per claim)

9.2 Final Verdict (Judge Output)

```
{
  "final_decision":   "The company should proceed with a phased B2B pivot...",
  "confidence_score": 82,
  "risk_level":       "Medium",
  "winning_agent":    "analyst",
  "key_reasons": [
    "B2B contracts provide 3x higher LTV than B2C",
    "Market data shows 18-month window before competition intensifies",
    "Current team expertise aligns with enterprise sales"
  ],
  "dissenting_view":  "Skeptic raised valid concerns about longer sales cycles",
  "explanation":      "The Judge weighed strong quantitative evidence from..."
}
```

9.3 Analytics Output

- Agent leaderboard with weighted scores and ranking badges
- Bar chart: per-metric breakdown (Logic / Evidence / Relevance / Persuasion)
- Line chart: score progression across rounds
- Token usage bar chart per agent
- Fact-check accuracy percentage and hallucination flags

10. Future Scope

- **Voice Input — Speech-to-text (Whisper API) for hands-free query entry**
  - Real-time transcription → immediate debate trigger
- **Web Search Integration — Live data via SerpAPI or Tavily**
  - Agents retrieve breaking news and live market data
- **More Agent Roles — Devil's Advocate, Ethicist, Financial Modeler**
  - Configurable agent roster per domain
- **Multi-Language Support — Translate queries and outputs**
  - Cross-language debate with unified scoring
- **Fine-Tuned Domain Models — LoRA-adapted models per domain**
  - Deeper domain expertise beyond prompt engineering
- **Human-in-the-Loop — Allow a human expert to join as an agent**
  - Real expert vs. AI agent debate mode
- **FastAPI Backend — Decouple Streamlit UI from engine**
  - REST API for programmatic debate triggering
- **Evaluation Framework — Automated accuracy benchmarking**
  - Compare decisions against historical ground truth